Contexto

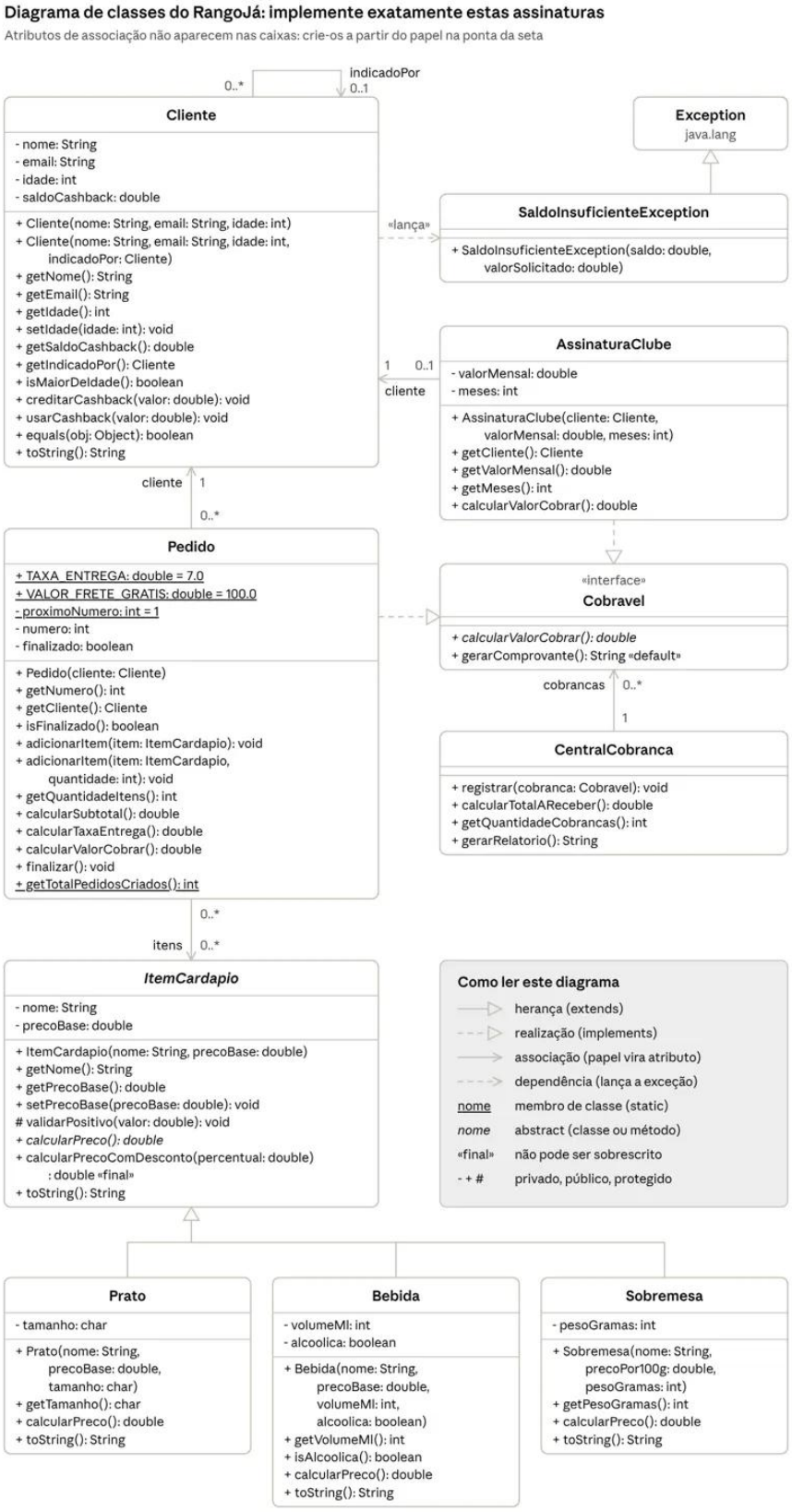
Você acabou de entrar no time do **RangoJá**, um app de delivery de Blumenau que promete acabar com a fila da cantina. O time de arquitetura já fechou o diagrama de classes; sua missão é implementar as classes e os métodos seguindo as histórias de usuário desta lista.

Como usar a lista:

- **O diagrama manda nas assinaturas.** Nomes de classes, atributos, métodos, parâmetros, tipos e visibilidade devem ser exatamente os do diagrama.
- **As histórias mandam no comportamento.** Elas dizem as regras de negócio e quando cada exceção deve ser lançada. O como fica com você.
- **Siga a ordem das etapas.** Cada etapa depende das anteriores.
-

Etapa	O que você constrói	Conteúdos praticados
1	Cliente e SaldoInsuficienteException	Unidade 01: encapsulamento, construtores, sobrecarga, this, exceções
2	Cardápio: ItemCardapio, Prato, Bebida, Sobremesa	Unidade 04: herança, abstract, protected, super, sobrescrita, final
3	Pedido	Unidades 01, 03 e 05: static, sobrecarga, associações, polimorfismo, instanceof
4	Cobravel, AssinaturaClube, CentralCobranca	Unidade 05: interfaces, método default, polimorfismo via interface
5	App com o main	Unidades 01 e 03: try/catch, finally, multi-catch, diagrama de objetos
6	Classes de teste	Unidade 02: plano de testes, assert, anotações do JUnit

Diagrama de classes



Leia as setas pela ponta: Pedido conhece o seu Cliente e os seus itens, mas Cliente não conhece os seus pedidos; Pedido e AssinaturaClube não têm parentesco, mas cumprem o mesmo contrato Cobravel.

Etapas e histórias de usuário
Cada história traz o pedido do usuário e as regras que sua implementação precisa respeitar. Quando a regra disser "lança", é exceção mesmo, com uma mensagem clara.
Etapa 1: cliente e cashback
US01. Cadastro. Como cliente, quero me cadastrar com nome, e-mail e idade, para poder fazer pedidos no app.

- Nome e e-mail são obrigatórios: nulo ou vazio lança IllegalArgumentException.
- O e-mail precisa conter @.
- Idade válida vai de 0 a 120. A validação fica no setIdade, e o construtor reaproveita o setter em vez de repetir o if.
- Todo cliente começa com saldo de cashback zerado.
- isMaiorDeIdade responde se o cliente tem 18 anos ou mais.

US02. Indique um amigo. Como cliente, quero informar quem me indicou, para que meu amigo ganhe R$ 10,00 de cashback.

- O segundo construtor recebe o cliente indicador, guarda a referência e credita R$ 10,00 no saldo dele.
- Indicador nulo nesse construtor lança IllegalArgumentException.
- Dica: um construtor pode chamar o outro com this(nome, email, idade).

US03. Usar o cashback. Como cliente, quero acumular e usar cashback, para pagar menos nos próximos pedidos.

- creditarCashback: valor menor ou igual a zero lança IllegalArgumentException.
- usarCashback: valor menor ou igual a zero lança IllegalArgumentException; valor maior que o saldo lança SaldoInsuficienteException; caso contrário, desconta do saldo.
- SaldoInsuficienteException é verificável (estende Exception), então quem a lança precisa declarar throws. A mensagem informa o saldo e o valor pedido, por exemplo: "Saldo insuficiente: disponível R$ 4,00, solicitado R$ 10,00".

US04. Identificação. Como atendente do suporte, quero comparar e exibir clientes de forma legível.

- Dois clientes são iguais (equals) quando têm o mesmo e-mail, ignorando maiúsculas e minúsculas.
- toString devolve "Nome <email>", por exemplo "Ana Souza ana@furb.br".

Etapa 2: cardápio
US05. Item de cardápio. Como restaurante, quero cadastrar itens com nome e preço base.

- ItemCardapio é abstrata: no cardápio não existe "item genérico", só pratos, bebidas e sobremesas.
- Nome obrigatório. Preço base precisa ser positivo: use o método protegido validarPositivo, que lança IllegalArgumentException para valor menor ou igual a zero. As subclasses também vão usá-lo.
- calcularPreco é abstrato: cada tipo de item calcula do seu jeito.
- toString devolve nome e preço final com duas casas, por exemplo "Xis Salada - R$ 39,00".

US06. Pratos em tamanhos. Como cliente, quero escolher o tamanho do prato, para comer o quanto tenho fome.

- Tamanhos aceitos: 'P', 'M' e 'G'. Qualquer outro lança IllegalArgumentException.
- P custa o preço base, M custa 30% a mais e G custa 60% a mais.
- toString aproveita o texto da superclasse e acrescenta o tamanho: "Xis Salada - R$ 39,00 (tamanho M)".

US07. Bebidas. Como cliente, quero pedir bebidas de volumes diferentes.

- O volume em ml precisa ser positivo.
- Bebida alcoólica tem acréscimo de R$ 3,00 pela embalagem com lacre; as outras custam o preço base.
- toString acrescenta o volume e, se for o caso, a marcação: "Chopp - R$ 15,00 (500 ml) [alcoólica]".

US08. Sobremesas por peso. Como cliente, quero comprar sobremesa pelo peso.

- Na sobremesa, o preço base é o preço de 100 g. O preço final é proporcional ao peso.
- O peso em gramas precisa ser positivo.
- toString acrescenta o peso: "Pudim - R$ 12,00 (150 g)".

US09. Promoções. Como restaurante, quero aplicar desconto percentual em qualquer item, sem que nenhuma subclasse consiga mudar essa regra.

- calcularPrecoComDesconto recebe um percentual de 0 a 100 (fora disso, IllegalArgumentException) e aplica sobre o resultado de calcularPreco.
- O método é final.
- Repare que ele chama calcularPreco sem saber de qual subclasse é o objeto. Guarde essa ideia para a etapa 3.

Etapa 3: pedidos

US10. Abrir pedido. Como cliente, quero abrir um pedido, para montar minha compra.

- Pedido sem cliente lança IllegalArgumentException.
- Cada pedido recebe um número sequencial automático (1, 2, 3...), controlado por proximoNumero, que é compartilhado por todos os pedidos. getTotalPedidosCriados informa quantos pedidos já foram criados.
- O pedido conhece o seu cliente, mas o cliente não conhece os seus pedidos: confira a navegabilidade no diagrama.

US11. Adicionar itens. Como cliente, quero colocar itens no pedido, inclusive vários iguais de uma vez.

- Item nulo lança IllegalArgumentException; pedido já finalizado lança IllegalStateException.
- Bebida alcoólica só entra se o cliente for maior de idade; caso contrário, IllegalArgumentException. Aqui você vai precisar de instanceof e downcasting.
- A versão com quantidade adiciona o mesmo item várias vezes. Quantidade menor que 1 lança IllegalArgumentException. Reaproveite a outra versão em vez de duplicar as regras.

US12. Quanto vou pagar. Como cliente, quero ver o valor do pedido antes de finalizar.

- calcularSubtotal soma os preços de todos os itens com um único laço e sem nenhum instanceof: cada item responde com o seu próprio calcularPreco.
- A taxa de entrega é TAXA_ENTREGA (R$ 7,00), e fica grátis quando o subtotal chega a VALOR_FRETE_GRATIS (R$ 100,00).
- O valor a cobrar é o subtotal mais a taxa de entrega.

US13. Finalizar. Como cliente, quero finalizar o pedido e ganhar cashback.

- Pedido sem itens ou já finalizado lança IllegalStateException.
- Ao finalizar, o cliente recebe 5% do subtotal como cashback e o pedido fica marcado como finalizado.

Etapa 4: cobranças

US14. RangoClube. Como cliente, quero assinar o clube do RangoJá por alguns meses.

- Cliente obrigatório, valor mensal positivo e pelo menos 1 mês; senão, IllegalArgumentException.

- O valor a cobrar é o valor mensal vezes a quantidade de meses. Assinaturas de 12 meses ou mais ganham 15% de desconto.
- AssinaturaClube não tem parentesco nenhum com Pedido e, mesmo assim, os dois são cobráveis.

US15. Contrato de cobrança. Como financeiro, quero tratar do mesmo jeito tudo que gera cobrança.
- Cobravel declara calcularValorCobrar; Pedido e AssinaturaClube implementam.
- gerarComprovante é um método default da interface e devolve "Valor a cobrar: R$ 64,00", usando calcularValorCobrar.

US16. Central de cobrança. Como financeiro, quero saber o total a receber.
- registrar aceita qualquer Cobravel; nulo lança IllegalArgumentException.
- calcularTotalAReceber soma tudo em um único laço, sem instanceof nem cast.
- gerarRelatorio monta um texto com uma linha por cobrança (usando gerarComprovante) e, no fim, a linha "Total a receber: R$ ...".

Etapa 5: juntando tudo no main

Crie a classe App simulando um dia no RangoJá. O programa nunca pode "morrer" com exceção: todo erro previsto é tratado e explicado ao usuário.

1. Monte o cardápio em um ArrayList<ItemCardapio> com pelo menos um item de cada tipo e exiba todos com um laço.
2. Cadastre a Ana (20 anos) e o Beto (17 anos), indicado pela Ana. Mostre o saldo de cashback da Ana.
3. Abra um pedido para o Beto e leia do teclado a quantidade de um prato. Se o usuário digitar algo que não é número ou uma quantidade inválida, avise e peça de novo até dar certo.
4. Tente colocar um chopp no pedido do Beto e trate o erro mostrando a mensagem da exceção.
5. Abra um pedido para a Ana e coloque as chamadas de adicionarItem e finalizar em um mesmo try, tratando IllegalArgumentException e IllegalStateException em um único catch (multi-catch). Depois tente finalizar o mesmo pedido de novo.
6. Faça a Ana tentar usar mais cashback do que tem. Trate a SaldoInsuficienteException e use finally para exibir o saldo atual, dando certo ou não.
7. Crie uma assinatura do RangoClube para a Ana, registre a assinatura e os pedidos finalizados na CentralCobranca e exiba o relatório.

Etapa 6: testes com JUnit

Cada classe de negócio ganha uma classe de teste paralela, com sufixo Test, na pasta test. Antes de escrever o código de teste, escreva o plano de testes de cada classe (caso, descrição, entrada e saída esperada).

- Use @BeforeEach para montar o contexto comum, como clientes e itens usados em vários casos.
- Para valores double, use assertEquals com limite de 0.001.
- Para exceções, use assertThrows com um lambda, inclusive para a SaldoInsuficienteException.
- Implemente os casos obrigatórios abaixo e acrescente pelo menos dois casos seus em cada classe de teste.

Classe de teste	Cenário	Resultado esperado
ClienteTest	Cliente com idade -1	IllegalArgumentException
ClienteTest	Cliente com e-mail "ana.furb.br"	IllegalArgumentException
ClienteTest	Beto criado com a Ana como indicadora	Saldo da Ana = 10,00
ClienteTest	Ana com saldo de 10,00 tenta usar 15,00	SaldoInsuficienteException
ClienteTest	Clientes com "ana@furb.br" e "ANA@FURB.BR"	equals verdadeiro
PratoTest	Prato com tamanho 'X'	IllegalArgumentException
PratoTest	Prato de 30,00, tamanho M, com 10% de desconto	35,10

PratoTest	Desconto de 120%	IllegalArgumentException
BebidaTest	Chopp de 12,00, 500 ml, alcoólico	15,00
SobremesaTest	Pudim de 8,00 por 100 g, com 150 g	12,00
PedidoTest	Xis Salada 30,00 (M) + Refri 6,00 (350 ml) + Pudim 8,00 por 100 g (150 g)	Subtotal 57,00, taxa 7,00, total 64,00
PedidoTest	Dois Xis de 40,00 tamanho G	Taxa 0,00, total 128,00
PedidoTest	Cliente de 17 anos adiciona um Chopp	IllegalArgumentException
PedidoTest	Adicionar com quantidade 0	IllegalArgumentException
PedidoTest	Finalizar pedido sem itens	IllegalStateException
PedidoTest	Finalizar pedido com subtotal de 57,00	Cashback do cliente aumenta 2,85
PedidoTest	Dois pedidos criados em sequência	Números consecutivos
CentralCobrancaTest	Pedido de 64,00 + assinatura de 19,90 por 12 meses	Total a receber 266,98

Atenção no teste de numeração: proximoNumero é static e não volta para 1 entre um teste e outro. Compare a diferença entre os números dos dois pedidos, não o valor absoluto.

Para ir além e entrega
Responda em poucas linhas, com as suas palavras:

1. Por que proximoNumero é static e numero não é?
2. Por que calcularPrecoComDesconto é final e calcularPreco é abstract? Um mesmo método poderia ser as duas coisas?
3. Se a navegação entre Pedido e Cliente fosse bidirecional, o que mudaria na classe Cliente?
4. Por que usarCashback precisa de throws e adicionarItem não precisa, mesmo os dois lançando exceções?

Checklist de entrega:

- Todas as classes do diagrama, com as mesmas assinaturas
- App executando os 8 passos da etapa 5 sem encerrar por exceção
- Classes de teste passando, com os casos obrigatórios e pelo menos dois casos seus por classe
- Respostas das quatro perguntas